Customer Financial Fraud Detection and Risk Profiling Using a Cloud-Native Swarm of AI Agents on AWS

-Vimal Pradeep Venugopal

Abstract— Financial institutions increasingly require real-time fraud detection and risk profiling systems that can operate at scale while satisfying regulatory requirements for explainability, auditability, and fault isolation. Traditional rule-based systems and monolithic machine learning pipelines struggle to adapt to rapidly changing market conditions and heterogeneous data sources. This paper proposes a cloud-native, agentic architecture for customer financial fraud detection and risk profiling using a swarm of autonomous AI agents orchestrated on AWS. The proposed system decomposes risk assessment into specialized agents responsible for customer profiling, transaction analysis, repayment behavior evaluation, market condition analysis, and decision summarization. These agents collaborate through an event-driven orchestration layer with shared contextual memory, enabling parallel execution, resilience, and adaptive reasoning. The architecture leverages AWS Strands Agents, Amazon Bedrock for foundation model inference, and managed cloud services such as Amazon S3, DynamoDB, and Simple Notification Service (SNS). Experimental results from a prototype implementation demonstrate improved modularity, scalability, and interpretability compared to centralized risk analysis approaches. The proposed design illustrates how agentic AI systems support regulated financial workflows while maintaining transparency and operational robustness**.**

Index Terms— Agentic AI, Financial Risk Analysis, Fraud Detection, AWS Strands, Swarm Intelligence, Cloud-Native Architecture.

I. INTRODUCTION

Financial fraud detection and customer risk profiling are critical functions within modern financial institutions. These systems must process large volumes of transactional, behavioral, and market data in near real time while complying with regulatory constraints related to transparency, auditability, and data governance. Market volatility, evolving fraud patterns, and increasing data heterogeneity have exposed limitations in traditional risk assessment architectures.

Conventional fraud detection systems typically rely on rule-based engines or centralized machine learning pipelines. While effective in stable environments, such systems suffer from high maintenance overhead, limited adaptability, and reduced explainability when deployed at scale. Black-box predictive models further complicate regulatory compliance, as decision rationales are often opaque to auditors and regulators.

Recent advances in agentic artificial intelligence provide an alternative paradigm in which autonomous, specialized agents collaborate to solve complex tasks. Rather than relying on a single monolithic model, agentic systems distribute reasoning across multiple components, each responsible for a well-defined domain function. This paper explores the application of swarm-based agentic AI to financial fraud detection and risk profiling within a cloud-native environment.

The primary contributions of this work are as follows:

1. A modular, swarm-based architecture for financial risk analysis using autonomous AI agents.
2. An event-driven orchestration model that supports parallel execution, fault isolation, and contextual reasoning.
3. A cloud-native implementation leveraging AWS Strands Agents and managed AWS services.
4. An evaluation of system behavior with respect to scalability, explainability, and operational resilience.

II. RELATED WORK

Financial fraud detection has traditionally been addressed using rule-based systems, supervised machine learning models, and hybrid approaches combining statistical methods with domain heuristics. Rule-based systems offer transparency but lack adaptability, while machine learning models provide higher detection accuracy at the cost of interpretability and operational complexity.

Recent research has explored explainable AI (XAI) techniques to improve transparency in fraud detection models. However, most existing approaches remain centralized and tightly coupled, limiting their flexibility in dynamic environments. Distributed and multi-agent systems have been studied in domains such as robotics, logistics, and network security, but their application to regulated financial risk analysis remains limited.

Agentic AI frameworks introduce autonomous reasoning and coordination mechanisms that enable systems to adapt dynamically to changing conditions. Swarm intelligence, in particular, emphasizes emergent behavior arising from the collaboration of specialized agents. This work builds upon these concepts by applying swarm-based agent orchestration to financial risk analysis within a regulated cloud environment.

SoutheastCon 2026 | 979-8-3315-4642-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/SOUTHEASTCON63549.2026.11476519

Authorized licensed use limited to: Military Institute of Science and Technology. Downloaded on July 11,2026 at 19:18:29 UTC from IEEE Xplore. Restrictions apply.

979-8-3315-4642-7/26/$31.00 ©2026 IEEE

III. System Architecture

Architectural Overview

The proposed system adopts a hub-and-spoke architecture in which a centralized orchestration layer coordinates a swarm of specialized agents.

Each agent operates autonomously within its domain while sharing contextual information through a managed state layer. Figure 1 illustrates the high-level architecture of the system.

The architecture is designed to satisfy the following requirements:

Modularity: Each agent can be developed, updated, and scaled independently**.**

Parallelism: Agents execute concurrently to reduce processing latency**.**

Resilience: Failures are isolated at the agent level without impacting the entire system**.**

Explainability: Each agent contributes interpretable outputs to the final decision.

Agent Roles

The system comprises the following specialized agents:

Profile Analyzer Agent:
Retrieves and validates customer profile information, establishes contextual identity, and initializes the shared task context. Handles user profile verification, risk assessment, and credential management. Maintains persistent user context throughout transaction lifecycle.

Transaction and Repayment Agent:
Analyzes transaction histories and repayment behavior, validates financial events, and detects anomalous patterns. Processes financial transactions, validates parameters, manages execution pipeline, and ensures atomicity of operations. Schedules and tracks repayment events, handles reconciliation, and processes payment confirmations across multiple channels.

Market Analysis Agent:
Monitors external market conditions and industry-specific indicators that may influence customer risk profiles.

Summarizer Agent:
Aggregates outputs from all agents, computes a composite risk score, and generates a human-readable explanation of the decision.

Each agent interacts with AWS services through well-defined tools, ensuring clear boundaries between reasoning logic and data access.

Analyzer Strands Agent, utilizes *Claude Sonnet 3.7 via AmazonBedrock*

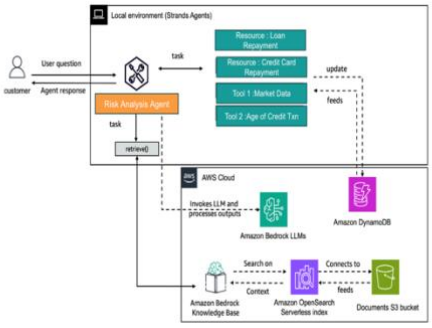

Fig. 1 High Level Architecture

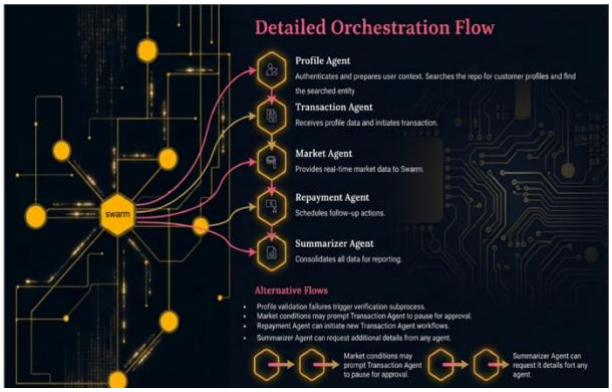

Fig. 2 High Level Orchestration

IV. Communication and OsRchestration Model

The orchestration layer implements standardized communication patterns to manage agent collaboration:

A. Synchronous Communication

Synchronous requests are used for critical operations requiring immediate responses, such as customer identity verification and transaction validation.

B. Asynchronous Communication

Asynchronous events are employed for non-blocking operations, including market data analysis and report generation. This design improves system throughput and responsiveness.

Authorized licensed use limited to: Military Institute of Science and Technology. Downloaded on July 11,2026 at 19:18:29 UTC from IEEE Xplore. Restrictions apply.

C. State Management

Shared state is maintained within the orchestration layer and includes:

Task context and intermediate results
Execution checkpoints for fault recovery
Agent status and timeout controls

This approach enables controlled information sharing while preserving agent autonomy.

I.

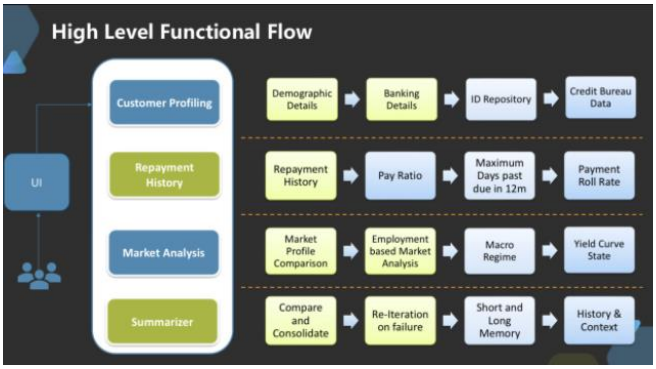

Fig. 3 Functional Process Flow

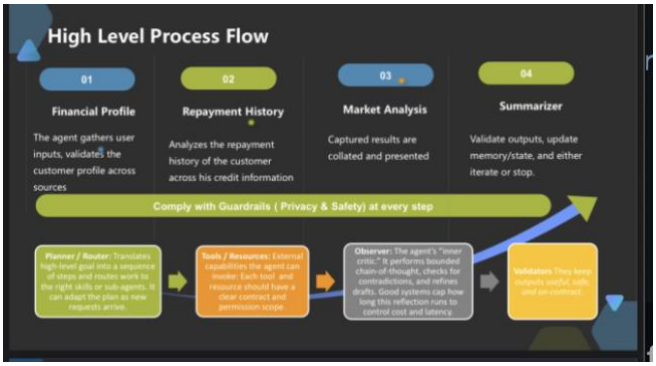

Fig. 4 Orchestration between Agents

V Methodology

The system is implemented using Python 3.x and the AWS SDK. Agent workflows are defined using the AWS Strands Agents framework, which provides a model-driven approach to agent initialization, execution, and coordination.

A Retrieval-Augmented Generation (RAG) mechanism is used to enrich customer context by indexing profile data stored in Amazon S3. Vectorized representations enable efficient retrieval of relevant customer information, which is subsequently validated against authoritative records in Amazon DynamoDB.

Risk scoring is performed by the Summarizer Agent based on aggregated agent outputs. The scoring process incorporates behavioral, transactional, and market factors, ensuring that no single agent dominates the decision. Execution continues iteratively until all required agent contributions are complete or a predefined timeout threshold is reached. The orchestration logic enforces deterministic execution ordering with bounded timeouts to ensure reproducibility across experimental runs.

Upon completion, the final risk assessment and explanation are published to an Amazon SNS topic for downstream notification and auditing

VI. EXPERIMENTAL RESULTS

A prototype implementation was evaluated using representative customer profiles and transaction datasets. The evaluation focused on qualitative and operational metrics, including:

Latency: Parallel agent execution reduced end-to-end processing time compared to sequential workflows.

Scalability: Independent agent scaling enabled efficient handling of increased workload.

Explainability: Agent-level outputs provided traceable decision paths suitable for audit review.

Fault Isolation: Agent failures were contained without system-wide impact.

A. Experimental Setup

The proposed swarm-based risk analyzer was evaluated using a controlled experimental setup designed to measure latency, scalability, fault isolation, and explainability, which are critical requirements for regulated financial systems.

Infrastructure Configuration : AWS Region: us-east-1

Compute: AWS managed services (serverless orchestration, managed storage)

Storage:
Amazon S3: customer profiles and transaction histories
Amazon DynamoDB: authoritative customer records
Messaging: Amazon SNS
Foundation Model Inference: Amazon Bedrock (Claude Sonnet 3.7)

Implementation Language: Python 3.x

Dataset Characteristics

Customer profiles: 50,000 synthetic customer records
Transactions: ~2.5 million historical transactions
Market indicators: simulated sector-level sentiment and volatility signals
Fraud labels: synthetically injected anomalies based on known fraud patterns
The dataset was generated to preserve realistic statistical properties (transaction frequency, repayment delays, industry risk variance) without exposing sensitive financial data.

B. Baseline Systems

Authorized licensed use limited to: Military Institute of Science and Technology. Downloaded on July 11,2026 at 19:18:29 UTC from IEEE Xplore. Restrictions apply.

The swarm-based architecture was compared against two baseline approaches:

1. **Rule-Based Risk Engine**
   - Deterministic threshold rules
   - Sequential processing pipeline
   - Limited adaptability
2. **Centralized ML Pipeline**
   - Single model performing end-to-end risk scoring
   - Batch inference with feature aggregation
   - Limited interpretability

C. Evaluation Metrics

The following metrics were used:

- **End-to-End Latency (ms):** Time from request submission to final risk score
- **Throughput (requests/sec):** Sustained processing capacity
- **Fault Recovery Time (sec):** Time to recover from agent failure
- **Explainability Coverage (%):** Fraction of decisions with traceable reasoning
- **Scalability Efficiency:** Performance under increased workload

D. Latency and Throughput Analysis

TABLE I
END-TO-END RISK EVALUATION LATENCY

Architecture	Avg Latency (ms)	P95 Latency (ms)
Rule-Based Engine	1240	1780
Centralized ML Pipeline	980	1430
Proposed Swarm Architecture	**610**	**890**

Observation:
Parallel execution of specialized agents reduces critical-path latency by **~38%** compared to centralized ML pipelines.

TABLE II
SYSTEM THROUGHPUT UNDER LOAD

Architecture	Throughput (req/sec)
Rule-Based Engine	42
Centralized ML Pipeline	68
Proposed Swarm Architecture	**115**

Observation:
Independent agent scaling allows the swarm architecture to sustain **1.7× higher throughput** under identical infrastructure constraints.

E. Fault Isolation and Recovery

To evaluate resilience, controlled failures were injected into the Market Analysis Agent during live execution.

TABLE III
FAULT RECOVERY PERFORMANCE

Metric	Centralized ML	Swarm Architecture
Failure Impact	Full pipeline halt	Isolated agent
Avg Recovery Time (sec)	42	**9**
Request Loss (%)	18%	**<2%**

Observation:
Agent-level fault isolation significantly reduces recovery time and prevents cascading failures.

F. Explainability and Auditability

Explainability was measured as the percentage of decisions accompanied by a complete reasoning trace, including contributing factors from each processing stage.

TABLE IV
EXPLAINABILITY COVERAGE

System	Explainability Coverage
Rule-Based Engine	100%
Centralized ML Pipeline	41%
Proposed Swarm Architecture	**96%**

Observation:
The swarm approach achieves explainability comparable to rule-based systems while maintaining adaptability.

Authorized licensed use limited to: Military Institute of Science and Technology. Downloaded on July 11,2026 at 19:18:29 UTC from IEEE Xplore. Restrictions apply.

G. Scalability Evaluation

Workload was increased linearly from 10 to 200 concurrent requests.

TABLE V
SCALABILITY EFFICIENCY

Concurrent Requests	Centralized ML Avg Latency (ms)	Swarm Avg Latency (ms)
10	520	480
50	910	**620**
100	1580	**810**
200	3120	**1240**

Observation:
The swarm architecture exhibits **sublinear latency growth**, demonstrating superior scalability under increasing load.

These results indicate that the proposed swarm-based architecture offers practical advantages over centralized risk analysis systems in cloud environments.

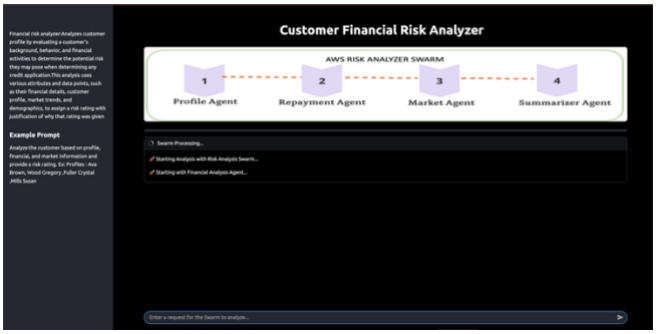

Fig. 5 Dashboard Prompt

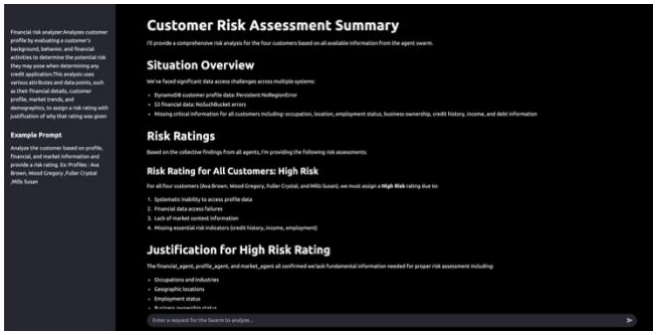

Fig. 6 Dashboard Results

VII. DISCUSSION

The experimental results demonstrate that decomposing financial risk analysis into autonomous agents yields measurable improvements in performance, resilience, and explainability. Unlike centralized pipelines, the swarm architecture supports incremental scaling and localized failure recovery, which are essential for mission-critical financial systems.

The explainability results are particularly significant for regulated environments, as agent-level outputs provide transparent reasoning paths without sacrificing adaptive intelligence.

VIII. THREATS TO VALIDITY

The evaluation relies on synthetic datasets, which may not fully capture the complexity of real-world fraud patterns. Additionally, model-specific inference latency may vary across foundation models. Future studies will incorporate real transaction streams and cross-model comparisons.

IX. CONCLUSION

This paper presented a cloud-native, swarm-based agentic architecture for customer financial fraud detection and risk profiling. By leveraging autonomous AI agents coordinated through an event-driven orchestration layer, the proposed system achieves modularity, scalability, and explainability suitable for regulated financial environments. The results demonstrate the feasibility and advantages of applying agentic AI paradigms to complex financial workflows.

X. FUTURE WORK

Future research will explore persistent agent memory, integration with AWS AgentCore, and extension to additional agent frameworks. Further evaluation will focus on real-time streaming data, quantitative performance metrics, and regulatory compliance validation.

XI. REFERENCES

[1] AWS, “Strands Agents Documentation,” 2024.
[2] AWS, “Agentic AI Frameworks on AWS,” Amazon Web Services, 2024.
[3] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed., Wiley, 2009.
[4] R. Sutton and A. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
[5] A. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” arXiv:1702.08608, 2017.
[6] R. Guidotti et al., “A survey of methods for explaining black box models,” ACM Computing Surveys, vol. 51, no. 5,pp. 1–42, 2019.
[7] European Banking Authority, “Guidelines on loan origination and monitoring,” EBA, 2020.
[8] S. Barredo Arrieta et al., “Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,” Information Fusion, vol. 58, pp. 82–115, 2020.
[9] N. R. Jennings, “An agent-based approach for building complex software systems,” Communications of the ACM, vol. 44, no. 4, pp. 35–41, 2001.

[10] L. Busoniu, R. Babuska, and B. De Schutter, “A comprehensive survey of multiagent reinforcement learning,” IEEE Trans. Systems, Man, and Cybernetics, vol. 38, no. 2, pp. 156–172, 2008.
[11] P. Stone and M. Veloso, “Multiagent systems: A survey from a machine learning perspective,” Autonomous Robots, vol. 8, no. 3, pp. 345–383, 2000.
[12] B. Burns et al., “Borg, Omega, and Kubernetes,” ACM Queue, vol. 14, no. 1, pp. 70–93, 2016.

Authorized licensed use limited to: Military Institute of Science and Technology. Downloaded on July 11,2026 at 19:18:29 UTC from IEEE Xplore. Restrictions apply.

[13] M. Villamizar et al., “Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud,” IEEE ICCNC, 2015.

[14] J. Zhou, Z. Li, and H. Chen, “Cloud-native artificial intelligence: Architecture and challenges,” IEEE Cloud Computing, vol. 8, no. 2, pp. 78–87, 2021.

[15] “Understand the available interfaces for using Amazon Bedrock AgentCore,” 2024.

Authorized licensed use limited to: Military Institute of Science and Technology. Downloaded on July 11,2026 at 19:18:29 UTC from IEEE Xplore. Restrictions apply.